

Tree Ensemble Methods

Bagging, Random Forests and Boosting

MATRA LAB

IIT Bhilai

September 16, 2026



Why Do We Need Ensemble Learning?

Decision Trees are Powerful

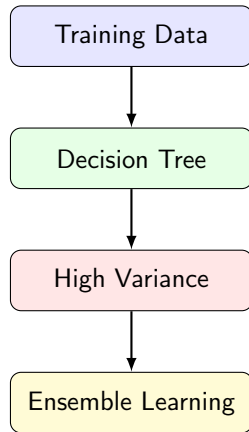
- Easy to understand and interpret
- Capture non-linear relationships
- Handle numerical and categorical features
- No assumptions about data distribution

Major Limitation A small change in the training data may generate a completely different tree.

High Variance $\implies$ Poor Generalization

Solution

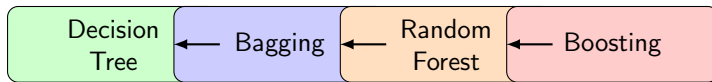
Instead of building one highly unstable decision tree, we combine many trees to obtain a more accurate and stable predictor.



Key Idea

Combine multiple trees instead of relying on only one.

Evolution of Tree-Based Ensemble Methods



Evolution

- Decision Tree
- Bagging
- Random Forest
- Boosting

Main Objective

- Improve Stability
- Reduce Variance
- Reduce Correlation
- Reduce Bias

Key Observation

Each method improves the shortcomings of the previous method while retaining the strengths of decision trees.

Bootstrap Sampling: Foundation of Bagging

Definition

Bootstrap Sampling is a resampling technique in which observations are sampled **with replacement** to generate multiple datasets having the same size as the original dataset.

Original Dataset

$$D = \{(x_i, y_i)\}_{i=1}^n$$

Generate Bootstrap Samples

$$D^{(1)}, D^{(2)}, D^{(3)}, \dots, D^{(B)}$$

Each bootstrap sample contains exactly **n** observations.

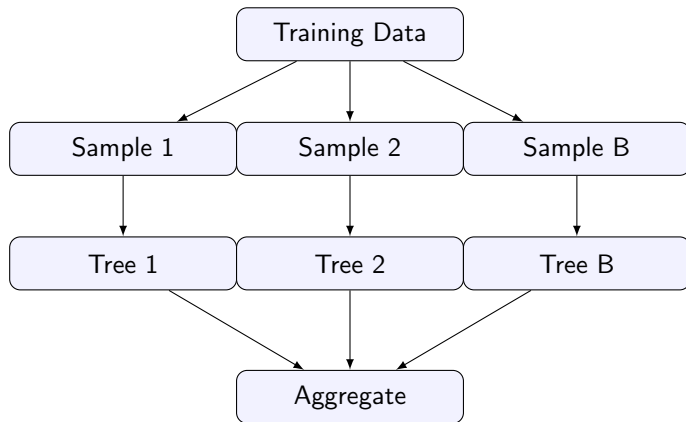
Important Property

Probability that an observation is **not selected** in a bootstrap sample is Hence, approximately **36.8%** of observations remain **Out-of-Bag (OOB)**.

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368$$

Bagging (Bootstrap Aggregating)

- Draw B bootstrap samples from the training data.
- Train a decision tree on each bootstrap sample.
- Aggregate the predictions (majority vote for classification, average for regression).



Bagging Algorithm

Bagging Procedure

For a training dataset

$$D = \{(x_i, y_i)\}_{i=1}^n,$$

Bagging constructs multiple bootstrap datasets, trains an independent Decision Tree on each dataset, and combines their predictions.

Aggregation Rule

- Regression : Average of all tree predictions.
- Classification : Majority vote among all trees.

Mathematics Behind Bagging

Bagging Predictor

Suppose B bootstrap datasets are generated and a Decision Tree is trained on each dataset. For a test observation x , the bagging predictor is

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

where

- $\hat{f}_b(x)$: Prediction of the b^{th} Decision Tree
- B : Total number of bootstrap trees

Regression

Final prediction is the average of all trees.

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

Classification

Each tree casts one vote.

The predicted class is the

Majority Vote

among all trees.

Why Does Bagging Reduce Variance?

Suppose each Decision Tree has prediction variance

$$\text{Var}(\hat{f}(x)) = \sigma^2$$

If all trees are **independent**, then averaging B trees gives

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)\right) = \frac{\sigma^2}{B}$$

Interpretation

- As the number of trees (B) increases, prediction variance decreases.
- Averaging multiple trees produces a more stable predictor.
- If $B \rightarrow \infty$, the variance approaches zero (ideal case).

Why Does Bagging Reduce Variance?

In practice, Decision Trees are **not independent**. They are trained from the same original dataset, so their predictions are correlated.

The variance of the bagging predictor becomes

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

where

ρ = Correlation between two Decision Trees

Interpretation

- If $\rho = 0$, trees are independent and bagging achieves maximum variance reduction.
- If $\rho = 1$, all trees are identical and bagging provides no improvement.
- Reducing the correlation between trees is the key motivation behind **Random Forests**.

Limitation of Bagging

Although Bagging significantly reduces the variance of Decision Trees, it suffers from one important limitation.

Decision Trees are often highly correlated.

Example

Suppose the dataset contains

- Age, Salary, Experience, Education, Credit Score

If **Salary** is the strongest predictor, then almost every tree will choose Salary as the first split.

Observation

Trees become very similar, resulting in a high correlation (ρ). Consequently, the variance reduction achieved by Bagging is limited.

Tree 1
Salary < 50K

Tree 2
Salary < 52K

Tree 3
Salary < 48K

Random Forest: Random Feature Selection

Key Idea

Instead of considering **all** predictors at every split, Random Forest randomly selects a subset of predictors and searches for the best split only among those variables.

Bagging

Features Available

- Age
- Salary
- Education
- Experience
- Credit Score

Best Split

⇒ Salary

Random Forest

Random Feature Subset

- Age
- Experience
- Credit Score

Best Split are Chosen only from these

Observation

- Every split uses a different random subset of features. Different trees grow differently.
- Tree correlation decreases. Lower correlation leads to better ensemble performance.

Random Forest Algorithm

Given a training dataset construct a Random Forest consisting of B Decision Trees.

$$D = \{(x_i, y_i)\}_{i=1}^n$$

construct a Random Forest consisting of B Decision Trees.

- ➊ Draw a bootstrap sample from the original dataset.
- ➋ Train an unpruned Decision Tree on the bootstrap sample.
- ➌ At each split, randomly select $m < p$ predictors from the total p predictors.
- ➍ Find the best split only among these m predictors.
- ➎ Repeat Steps 1–4 until B trees are grown.
- ➏ Aggregate predictions:
 - Regression : Average of all tree predictions
 - Classification : Majority voting

Random Forest = Bootstrap + Random Feature Selection + Aggregation

Choosing the Number of Features (m)

Suppose the dataset contains P predictor variables.

Instead of considering all p predictors at every split, Random Forest randomly selects only $m < p$ predictors and searches for the best split among them.

Problem Type	Typical Choice of m
Classification	$m = \sqrt{p}$
Regression	$m \approx \frac{p}{3}$

Effect of Choosing m

- Small $m \rightarrow$ More randomness, lower tree correlation.
- Large $m \rightarrow$ Trees become similar to Bagging.
- Selecting an appropriate m improves prediction accuracy.

Out-of-Bag (OOB) Error Estimation

During bootstrap sampling, some observations are **not selected** for training a particular Decision Tree. These observations are called **Out-of-Bag (OOB)** samples.

The probability that an observation is **not selected** in a bootstrap sample is

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368$$

Hence, approximately **36.8%** of the observations are Out-of-Bag.

OOB Error Estimation

- 1 Predict each OOB observation using only the trees in which it was not used for training.
- 2 Aggregate these predictions.
- 3 Compare the predicted and true values to estimate the prediction error.

$$\text{OOB Error} \approx \text{Test Error}$$

Variable Importance in Random Forest

Random Forest can estimate the importance of each predictor by measuring its contribution to the prediction accuracy.

Two commonly used measures are:

① Mean Decrease in Impurity (MDI)

- Based on the reduction in impurity (RSS, Gini Index, etc.).
- Variables producing larger impurity reduction are considered more important.

② Permutation Importance

- Randomly shuffle the values of one predictor.
- Measure the decrease in prediction accuracy.
- Larger decrease $\Rightarrow$ More important feature.

Large Accuracy Drop $\Rightarrow$ Important Predictor

Applications

- Feature Selection
- Model Interpretation

Why Do We Need Boosting?

Random Forest improves prediction accuracy by averaging many Decision Trees.

However,

- Every tree is built independently.
- All trees have equal importance.
- Previous mistakes are ignored.

Key Question

Can a new tree learn from the mistakes made by the previous trees?

Boosting = Sequential Learning + Error Correction

Instead of averaging independent trees, Boosting builds trees sequentially, where each new tree attempts to improve the previous model.

Residual Learning

Suppose the first Decision Tree predicts

$$\hat{y}$$

instead of the true target

$$y.$$

The prediction error (Residual) is

$$r = y - \hat{y}$$

Instead of learning the original target again, the next Decision Tree learns the **Residuals** generated by the previous tree.

Tree 1

→

Residual

→

Tree 2

Additive Model of Boosting

Boosting constructs the final model by adding together many weak learners. After the first tree,

$$F_1(x) = f_1(x)$$

After the second tree,

$$F_2(x) = f_1(x) + f_2(x)$$

After M trees,

$$F_M(x) = \sum_{m=1}^M f_m(x)$$

Unlike Bagging,

- Bagging averages predictions.
- Boosting adds prediction corrections.

Why Learn Residuals?

Suppose the true target is

$$y$$

and the current model predicts

$$\hat{y}$$

The prediction error (Residual) is

$$r = y - \hat{y}$$

Instead of learning the target again,
the next tree learns only the residual.

$$y = \hat{y} + r$$

Residual = Negative Gradient

Consider

$$L = \frac{1}{2}(y - F(x))^2$$

Differentiate with respect to

$$F(x)$$

$$\frac{\partial L}{\partial F} = -(y - F(x))$$

Therefore

$$-\frac{\partial L}{\partial F} = y - F(x) = r$$

Learning Rate (Shrinkage)

Instead of adding the complete correction,
Boosting updates the model gradually.

$$F_m(x) = F_{m-1}(x) + \lambda h_m(x)$$

$$0 < \lambda \leq 1$$

Smaller λ More trees Better generalization

Forward Stagewise Additive Modeling

After the first tree

$$F_1(x) = f_1(x)$$

After the second tree

$$F_2(x) = F_1(x) + \lambda f_2(x)$$

After M trees

$$F_M(x) = \sum_{m=1}^M \lambda f_m(x)$$

One Tree at a Time

→

One Correction at a Time

Why is it called Gradient Boosting?

Residual Learning

$$r = y - F(x)$$

For Squared Error Loss,

$$-\frac{\partial L}{\partial F} = y - F(x) = r$$

For a general loss function,

$$\text{Residual} \neq -\frac{\partial L}{\partial F}$$

Gradient Boosting fits the **Negative Gradient of the Loss Function** instead of simply fitting residuals.

Gradient Boosting Algorithm

- 1 Initialize the model

$$F_0(x)$$

- 2 Compute the negative gradients (pseudo-residuals).
- 3 Fit a weak Decision Tree to these pseudo-residuals.
- 4 Update the model

$$F_m(x) = F_{m-1}(x) + \lambda h_m(x)$$

- 5 Repeat until M trees are constructed.

Final Model

$$F_M(x) = \sum_{m=1}^M \lambda h_m(x)$$

Important Hyperparameters

The performance of Gradient Boosting depends on three major hyperparameters.

Learning Rate (λ)	Controls the contribution of each tree.
Number of Trees (M)	More trees generally improve accuracy but increase training time.
Tree Depth	Controls the complexity of each weak learner.

Typical values

$$\lambda = 0.1, \quad M = 100\text{--}1000, \quad \text{Depth} = 1\text{--}5$$

Comparison of Ensemble Methods

Property	Bagging	Random Forest	Boosting
Training	Parallel	Parallel	Sequential
Bootstrap	Yes	Yes	No
Random Features	No	Yes	No
Main Goal	Reduce Variance	Reduce Correlation	Reduce Bias
Prediction	Average	Average	Additive Model
Base Learners	Independent	Independent	Dependent

Decision Tree → Bagging → Random Forest → Gradient Boosting